TP 1 : Acquisition, Représentation et Transformations d'Images Numériques

🎯 Objectifs :

À la fin de ce TP, l'étudiant sera capable de :

- Comprendre la représentation matricielle d'une image
- Manipuler les images en niveaux de gris et en couleur (RGB)
- Étudier l'effet de l'échantillonnage et de la quantification
- Analyser et interpréter un histogramme
- Implémenter des transformations ponctuelles
- Appliquer des transformations locales (convolution)

Préparation de l'environnement:

Bibliothèques à installer:

```
pip install opencv-python numpy matplotlib                    </> CMD
```

Bibliothèques à importer:

```
import cv2              # Traitement d'image                  </> Python
import numpy as np      # Calcul scientifique (matrices)
import matplotlib.pyplot as plt  # Affichage graphique
```

Partie 1 : Acquisition et Représentation:

Comprendre qu'une image est une matrice 2D (gris) ou 3D (couleur).

Fonction	Rôle
`cv2.imread()`	Lecture d'image
`cv2.cvtColor()`	Conversion RGB → Gris
`img.shape`	Dimensions
`img.dtype`	Type numérique
`plt.imshow()`	Affichage de l'image
`cv2.split()`	Séparer canaux
`plt.subplot(x,y,i)`	Affichage multiple
`plt.show()`	Affichage de la figure

Exercice 1 : Lecture et analyse d'une image

1. Lire une image couleur "rgb.png".
   - Afficher : L'image originale; Sa taille (M × N × 3); Son type
2. Afficher chaque canal (R,G,B) séparément par la fonction `subplot()`

3. Lire les deux images couleur " usthbc.jpg"." usthbs.jpg".
4. Convertir les images en niveaux de gris.

```
Img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

5. Afficher et vérifier que l'image en gris est une matrice M × N.

Partie 2 : Numérisation:

Exercice 2 : Échantillonnage (Réduction de résolution)

Objectif: Étudier la résolution spatiale.

```
Img small = cv2.resize(Img gray, None, fx=0.5, fy=0.5)
```

1. Réduire la résolution de l'image :
   - 1/2 ; 1/4 et 1/8
2. Afficher les résultats séparément par la fonction `subplot()`
3. Comparer la perte de détails.

Exercice 3 : Quantification

```
Img_4= (Img_gray // 16) * 16   # 4 bits
Img 2 = (Img gray // 64) * 64   # 2 bits
```

Objectif : Étudier la résolution en niveaux de gris.

1. Convertir l'image de 8 bits en :
   - 4 bits ; 2 bits ; 1 bit
2. Afficher les résultats séparément par la fonction `subplot()`

Partie 3 : Amélioration

Pour chaque exercice, afficher l'image origine, l'image résultat, et les histogrammes

Calcul et affichage de l'histogramme

```
hist = cv2.calcHist([Img_gray], [0], None, [256], [0,256])
plt.plot(hist)
```

Exercice 5 : Étirement d'histogramme

1. Implémenter l'expansion dynamique :

$$I' = \frac{I - I_{min}}{I_{max} - I_{min}} \times 255$$

```
Imin = np.min(Img_gray)
Imax = np.max(Img_gray)
Img_out = (Img_gray - Imin) * 255 / (Imax - Imin)
Img_out = Img_out.astype(np.uint8)
```

Exercice 7 : Égalisation d'histogramme

```
Img_equalized = cv2.equalizeHist(Img_gray)
```

Exercice 9 : Seuillage

```
Img_bin = cv2.threshold(Img_gray, 127, 255, cv2.THRESH_BINARY)
```

Exercice 10 : Gamma

```
gamma = 0.5
gamma_corrected = np.array(255*( Img_gray /255)**gamma, dtype='uint8')
```

Exercice 11 : Logarithmique

```
c = 255 / np.log(1 + np.max(Img_gray))
log_img = c * np.log(1 + Img_gray)
log_img = np.array(log_img, dtype='uint8')
```